MATERI PERKULIAHAN

DESAIN ARSITEKTUR PERANGKAT LUNAK

Prinsip desain • Pola arsitektur • Komunikasi • Cloud • Evaluasi

Tujuan: Mahasiswa mampu menerapkan konsep algoritma pemrograman berbasis komputasi untuk menganalisis model interaktif dalam konteks informatika.

Tujuan Pembelajaran

Kompetensi yang dituju setelah mempelajari materi

Merupakan proses merancang struktur dasar suatu sistem perangkat lunak dengan menentukan komponen-komponen utama, hubungan antar-komponen, pembagian fungsi, serta bagaimana setiap komponen saling berkomunikasi untuk membentuk sebuah sistem yang utuh.

MEMAHAMI

Konsep, prinsip, dan karakteristik berbagai model arsitektur perangkat lunak.

MENGANALISIS

Struktur komponen, aliran data, interaksi, dan kebutuhan sistem.

MENERAPKAN

Konsep algoritma dan komputasi untuk merancang serta mengevaluasi solusi.

Fokus pembelajaran: bukan menghafal istilah, tetapi memahami alasan suatu arsitektur dipilih dan bagaimana komponennya bekerja.

1. Tujuan Desain Arsitektur Perangkat Lunak

1. Mengatur struktur sistem: Menentukan bagian-bagian utama yang membentuk aplikasi.
2. Membagi tanggung jawab: Menentukan tugas dan fungsi setiap komponen.
3. Mengatur komunikasi: Menentukan bagaimana komponen saling bertukar data.
4. Mempermudah pengembangan: Membantu programmer mengembangkan sistem secara terstruktur.
5. Mempermudah pemeliharaan: Memudahkan perbaikan dan pengembangan sistem di masa depan.
6. Menjamin kualitas sistem: Mendukung keamanan, kinerja, keandalan, dan skalabilitas aplikasi.

2. Elemen Utama Arsitektur Perangkat Lunak

Elemen	Penjelasan
Komponen	Bagian utama pembentuk aplikasi.
Modul	Bagian program yang memiliki fungsi tertentu.
Interface	Penghubung komunikasi antar-komponen.
Database	Tempat penyimpanan dan pengelolaan data.
Connector	Mekanisme komunikasi antar-komponen.
Deployment	Cara komponen ditempatkan dan dijalankan pada infrastruktu

Peta Materi

Sepuluh bahasan utama dalam desain arsitektur perangkat lunak

- Prinsip desain
- Monolith & Microservices
- Layered Architecture
- MVC Pattern
- Client-Server & P2P
- Event-Driven Architecture
- Service-Oriented Architecture
- Cloud-Native
- Evaluasi arsitektur
- Dokumentasi arsitektur

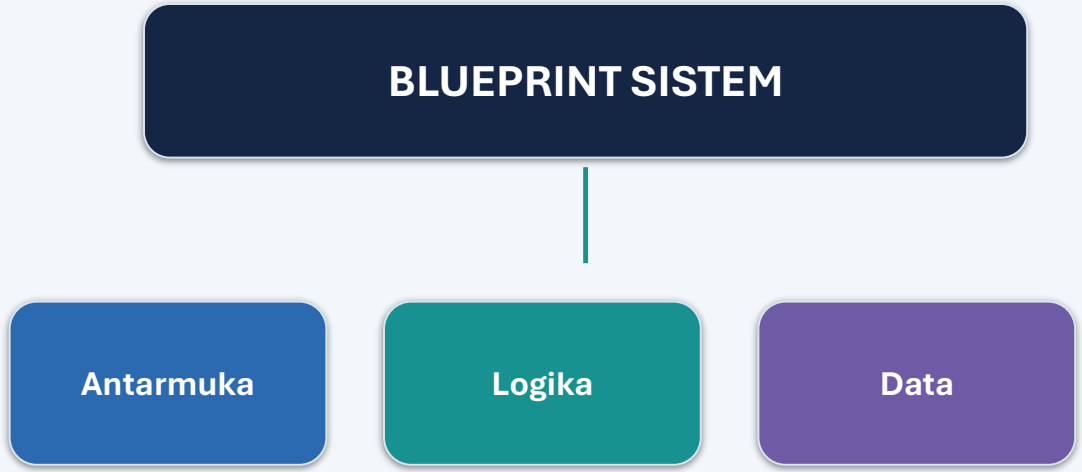
Alur: memahami prinsip → mengenali model → merancang interaksi → mengevaluasi dan mendokumentasikan.

Apa Itu Arsitektur Perangkat Lunak?

Rancangan struktur dasar sebuah sistem perangkat lunak

Arsitektur perangkat lunak menjelaskan komponen sistem, tanggung jawabnya, hubungan antarkomponen, dan cara komponen berkomunikasi.

- Struktur dan batas komponen
- Aliran data dan kontrol
- Antarmuka dan komunikasi
- Teknologi serta lingkungan eksekusi



Analogi: Blueprint bangunan menentukan ruang, fungsi, hubungan, dan jalur layanan sebelum pembangunan.

Mengapa Arsitektur Penting?

Keputusan arsitektur memengaruhi kualitas sistem sepanjang siklus hidup

Terstruktur

Komponen memiliki tanggung jawab jelas

Mudah dipelihara

Perubahan dan perbaikan lebih terarah

Skalabel

Kapasitas dapat dikembangkan sesuai beban

Aman dan andal

Risiko dan akses dapat dikelola

Arsitektur yang tepat membantu menyeimbangkan kebutuhan pengguna, biaya, kinerja, keamanan, dan kemampuan tim.

Hubungan Arsitektur, Algoritma, dan Komputasi

Empat unsur yang saling melengkapi



Contoh pembelian online: arsitektur membagi modul; algoritma memeriksa stok dan menghitung total; program menjalankan instruksi; komputasi menghasilkan transaksi.

1. **Arsitektur perangkat lunak** merupakan gambaran mengenai bagaimana sebuah aplikasi dirancang dan bagaimana bagian-bagian di dalamnya saling berhubungan.

2. Algoritma merupakan urutan langkah-langkah logis dan sistematis untuk menyelesaikan suatu permasalahan, sertamenjelaskan bagaimana setiap komponen melakukan pekerjaannya.

3. Program adalah implementasi algoritma menggunakan bahasa pemrograman sehingga komputer dapat menjalankan instruksi yang diberikan.

4. Komputasi merupakan proses pengolahan data menggunakan sumber daya komputasi berdasarkan instruksi program.

Prinsip Desain: Tiga Fondasi

Membentuk perangkat lunak yang terstruktur dan mudah dikelola

ABSTRAKSI

Menyederhanakan kompleksitas dengan menampilkan fungsi penting dan menyembunyikan detail internal.

MODULARITAS

Membagi sistem menjadi modul-modul kecil dengan tanggung jawab tertentu.

ENKAPSULASI

Menggabungkan data dan fungsi serta membatasi akses langsung ke data internal.

Ringkas: menyederhanakan • membagi • melindungi

Abstraksi (Abstraction)

Menampilkan hal penting, menyembunyikan detail yang tidak perlu



Contoh ATM:
Pengguna memilih tarik tunai, memasukkan PIN, dan menerima uang tanpa perlu memahami komunikasi jaringan atau proses database.

Contoh kode:
Fungsi hitung_total(harga, jumlah) menyembunyikan detail perkalian dari pemanggil fungsi.

Manfaat: mengurangi kompleksitas dan membuat sistem lebih mudah dipahami.

Modularitas (Modularity)

Membagi aplikasi menjadi bagian dengan fungsi khusus



Setiap modul menangani tanggung jawab tertentu. Modul dapat dikembangkan atau diuji dengan fokus pada fungsinya.

Manfaat: reuse, pembagian kerja tim, pelacakan kesalahan, dan pemeliharaan menjadi lebih mudah.

Prinsip penting: setiap modul sebaiknya memiliki tanggung jawab yang jelas dan keterikatan yang terkontrol.

Enkapsulasi (Encapsulation)

Melindungi data melalui antarmuka atau metode yang ditentukan



Contoh rekening bank:
Saldo tidak diubah langsung. Perubahan dilakukan melalui metode setor atau tarik yang memeriksa aturan transaksi.

Dalam pemrograman berorientasi objek, data dan metode pengelolanya berada dalam satu kelas; akses internal dibatasi.

Manfaat: menjaga integritas data dan mencegah perubahan yang tidak sesuai aturan.

Ilustrasi Prinsip dalam Kode Python

Contoh sederhana untuk menghubungkan teori dan implementasi

ABSTRAKSI

```
def hitung_total(harga, jumlah):
    return harga * jumlah

# Pemanggil cukup menggunakan
fungsi
```

MODULARITAS

```
def tambah_barang():
    ...

def hitung_bayar():
    ...

def cetak_struk():
    ...

# Tugas dipisah per fungsi
```

ENKAPSULASI

```
class Rekening:
    def __init__(self, saldo):
        self.__saldo = saldo

    def lihat_saldo(self):
        return self.__saldo

# Akses melalui metode
```

4. Perbedaan Ketiga Prinsip

Ketiga prinsip ini memiliki tujuan yang berbeda, tetapi saling melengkapi.

Prinsip	Fokus utama	Tujuan
Abstraksi	Menyederhanakan proses	Pengguna tidak perlu memahami detail implementasi.
Modularitas	Membagi program	Setiap bagian memiliki tanggung jawab yang jelas.
Enkapsulasi	Mengatur akses data	Data dikelola melalui mekanisme yang ditentukan.

Ilustrasi konsep; aplikasi nyata tetap memerlukan validasi dan penanganan kesalahan.

Model Arsitektur: Gambaran Umum

Setiap model menjawab kebutuhan struktur atau interaksi yang berbeda

Monolith

Satu aplikasi terpadu

Microservices

Layanan independen

Layered

Lapisan tanggung jawab

MVC

Model, View, Controller

Client–Server

Layanan terpusat

P2P

Antarpeer

EDA

Dipicu event

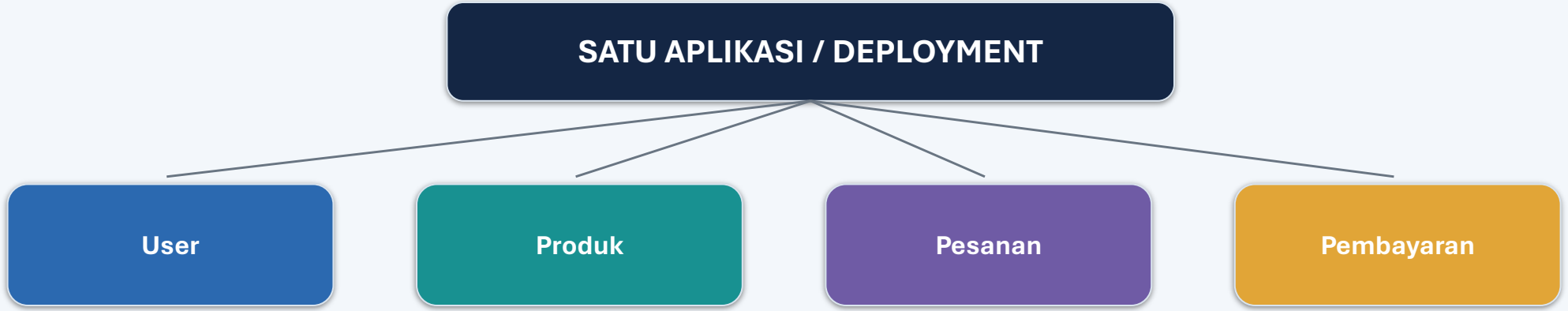
SOA

Integrasi layanan

Model dapat dikombinasikan sesuai kebutuhan; tidak ada satu arsitektur yang terbaik untuk semua kasus.

Monolithic Architecture

Seluruh fungsi utama dibangun sebagai satu aplikasi



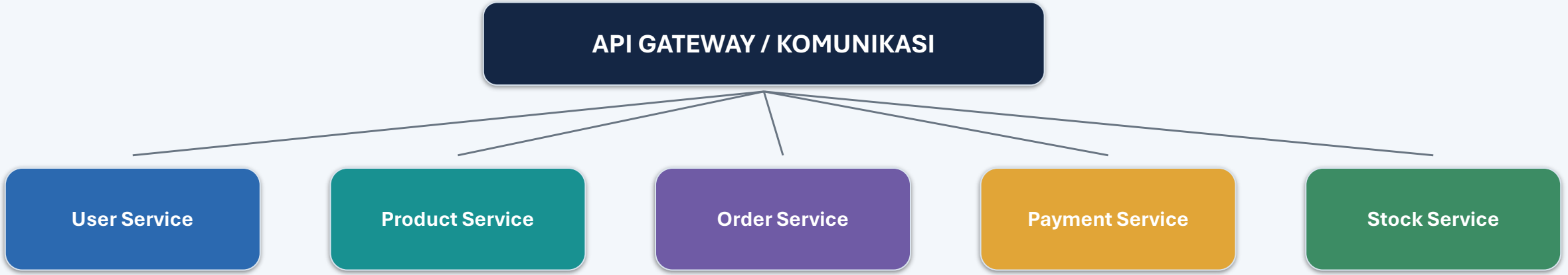
Kelebihan:
- Mudah dimulai dan diuji sebagai satu aplikasi
- Komunikasi internal relatif sederhana

Keterbatasan:
- Perubahan dapat berdampak luas
- Skalabilitas komponen secara terpisah terbatas

Cocok untuk aplikasi yang ruang lingkup dan kompleksitasnya masih terkendali.

Microservices Architecture

Aplikasi dibagi menjadi layanan kecil yang relatif independen



Kelebihan:
- Layanan dapat dikembangkan dan dirilis relatif independen
- Skalabilitas per layanan lebih fleksibel

Tantangan:
- Komunikasi jaringan dan monitoring lebih kompleks
- Konsistensi data dan keamanan perlu dirancang

Microservices sesuai untuk sistem besar atau berkembang jika tim siap mengelola kompleksitasnya.

Monolith vs Microservices

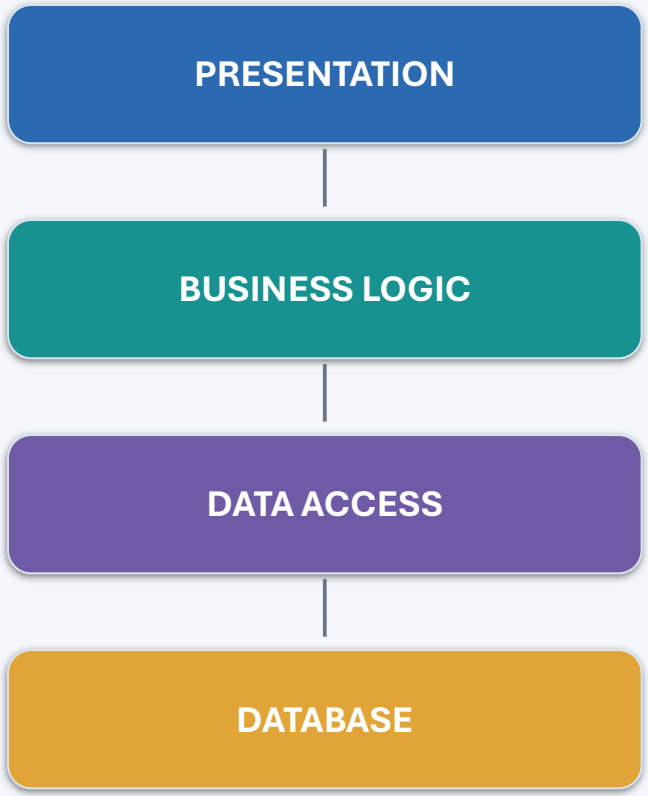
Perbandingan untuk membantu menentukan pilihan

Aspek	Monolith	Microservices
Struktur	Satu aplikasi	Banyak layanan
Rilis	Umumnya bersama	Dapat independen
Skalabilitas	Aplikasi secara luas	Per layanan
Kompleksitas	Relatif rendah	Lebih tinggi
Kesesuaian	Sederhana/menengah	Kompleks/bertumbuh

Keputusan arsitektur harus mempertimbangkan kebutuhan, biaya, kompetensi, dan operasi.

Layered Architecture (N-Tier)

Memisahkan sistem berdasarkan lapisan tanggung jawab



Antarmuka: menerima input dan menampilkan hasil

Aturan bisnis: validasi, perhitungan, keputusan

Akses: membaca dan mengubah data

Penyimpanan data

Contoh umum: 3-tier memisahkan presentation, business logic, dan data; N-tier dapat memiliki lapisan tambahan.

Contoh Alur Layered: Proses Login

Setiap lapisan menjalankan tanggung jawab yang berbeda



Hasil pemeriksaan dikembalikan melalui lapisan aplikasi hingga ditampilkan kepada pengguna. Pemisahan ini membantu perubahan tampilan tanpa mengubah aturan bisnis.

Manfaat: struktur jelas, pemeliharaan lebih mudah, dan tanggung jawab tidak bercampur.

Memahami N-Tier Architecture

Jumlah tier menyesuaikan kebutuhan pemisahan dan deployment

2-TIER

Client / aplikasi

Database

Aplikasi berkomunikasi relatif langsung dengan database.

3-TIER

Presentation

Business Logic

Database

Pemisahan tiga tanggung jawab utama.

N-TIER

Beberapa lapisan tambahan

Misalnya API, layanan integrasi, cache, dan data

Sesuai kompleksitas sistem.

Layer adalah pemisahan logis; tier sering merujuk pada pemisahan fisik atau deployment. Istilah dapat berbeda menurut konteks.

Model-View-Controller (MVC)

Pola desain yang memisahkan data, tampilan, dan kendali



Contoh MVC: Mahasiswa Melihat Nilai

Interaksi pengguna diproses melalui tiga bagian

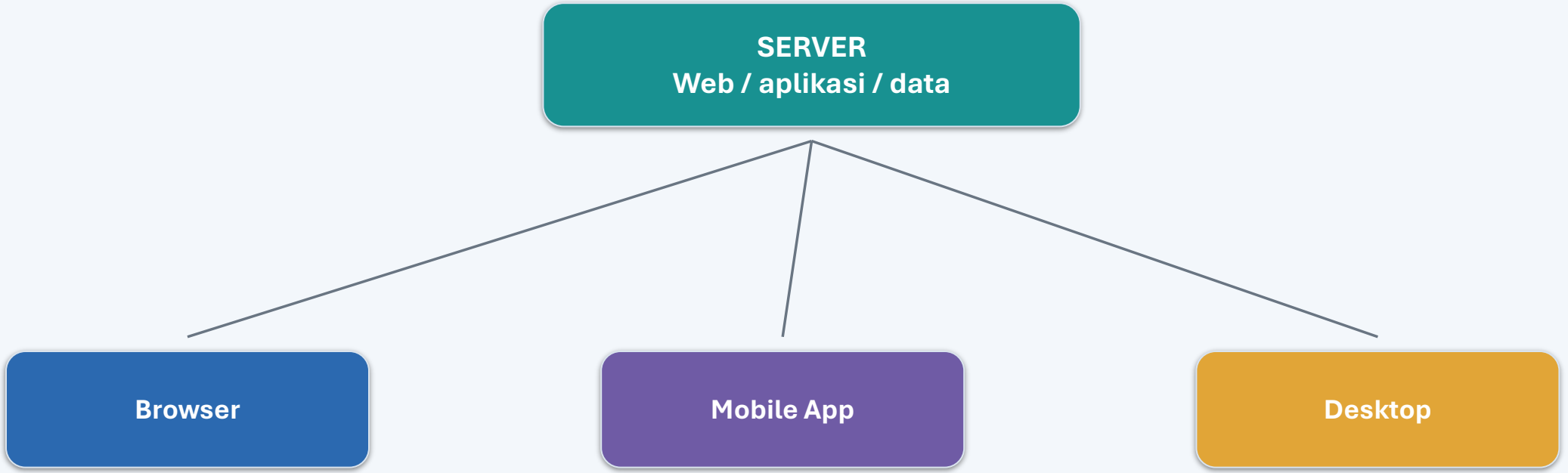


Pemisahan MVC memungkinkan tampilan diperbarui tanpa harus menulis ulang seluruh logika pengelolaan data.

Contoh framework yang menggunakan MVC atau variasinya: Laravel, Ruby on Rails, dan ASP.NET Core MVC.

Client–Server Architecture

Client meminta layanan; server memproses dan memberikan respons



Kelebihan: pengelolaan data dan layanan terpusat; kebijakan akses dapat dikelola di server.

Tantangan: ketergantungan pada server, kapasitas, jaringan, dan potensi titik kegagalan.

Alur Permintaan Client–Server

Contoh ketika pengguna membuka halaman website

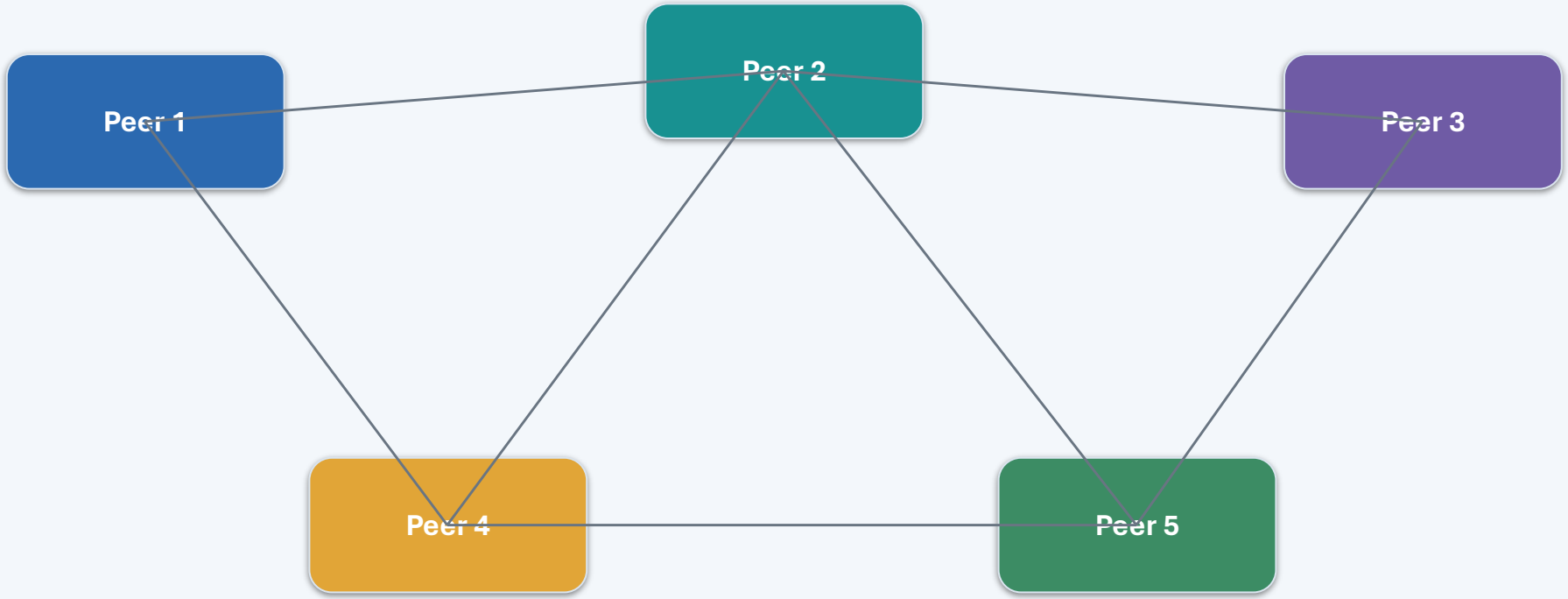


Komunikasi lazim menggunakan protokol jaringan seperti HTTP/HTTPS. Respons dapat berupa halaman, data JSON, atau file.

Contoh: browser sebagai client dan web server sebagai penyedia layanan.

Peer-to-Peer (P2P)

Setiap peer dapat menjadi peminta sekaligus penyedia sumber daya



Kelebihan: sumber daya tersebar dan tidak selalu memerlukan server pusat.

Tantangan: keamanan, ketersediaan peer, dan konsistensi lebih sulit dikelola.

Client–Server vs Peer-to-Peer

Perbedaan peran dan pengelolaan komunikasi

CLIENT–SERVER

- Struktur relatif terpusat
- Server menyediakan layanan
- Client mengirim permintaan
- Contoh: website, sistem akademik, perbankan

PEER-TO-PEER

- Struktur terdistribusi
- Peer dapat meminta dan menyediakan layanan
- Tidak selalu bergantung pada server pusat
- Contoh: berbagi file P2P

Pemilihan ditentukan kebutuhan kontrol, distribusi sumber daya, skala, dan keamanan.

Event-Driven Architecture (EDA)

Kejadian (event) menjadi pemicu proses lanjutan

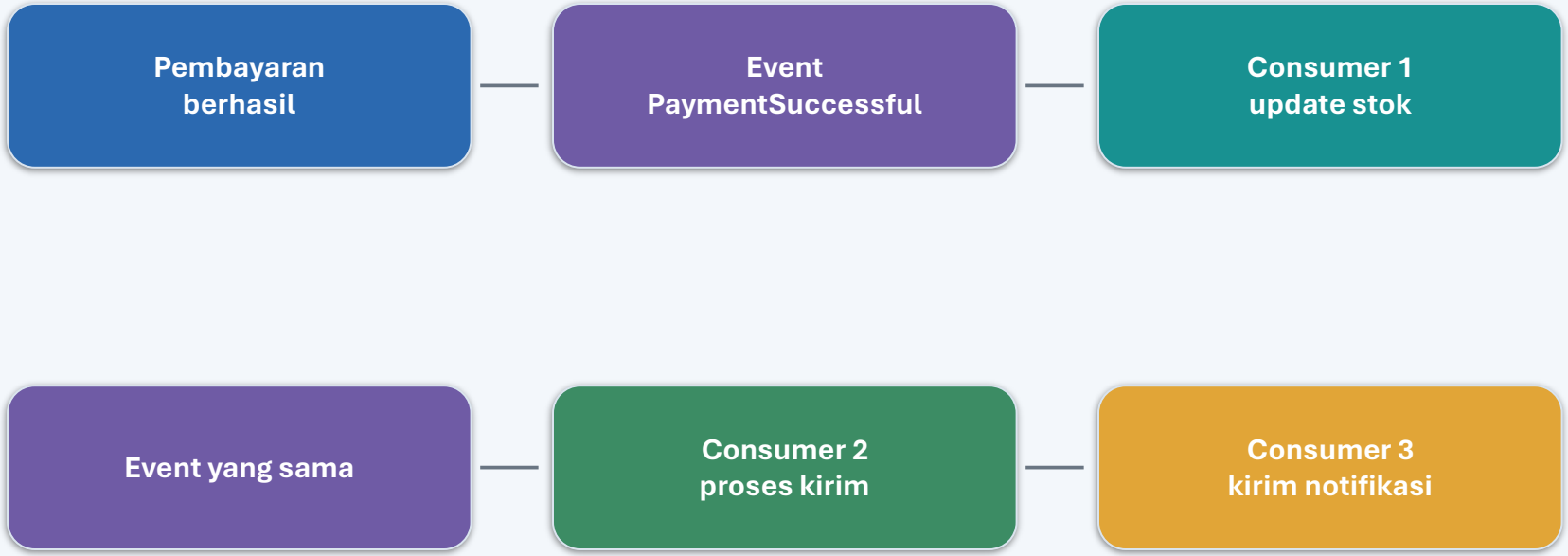


Contoh: PaymentSuccessful memicu pembaruan stok, proses pengiriman, dan notifikasi.

Komponen: Event Producer • Event Broker • Event Consumer. Proses dapat berjalan asynchronous.

Contoh EDA: Setelah Pembayaran Berhasil

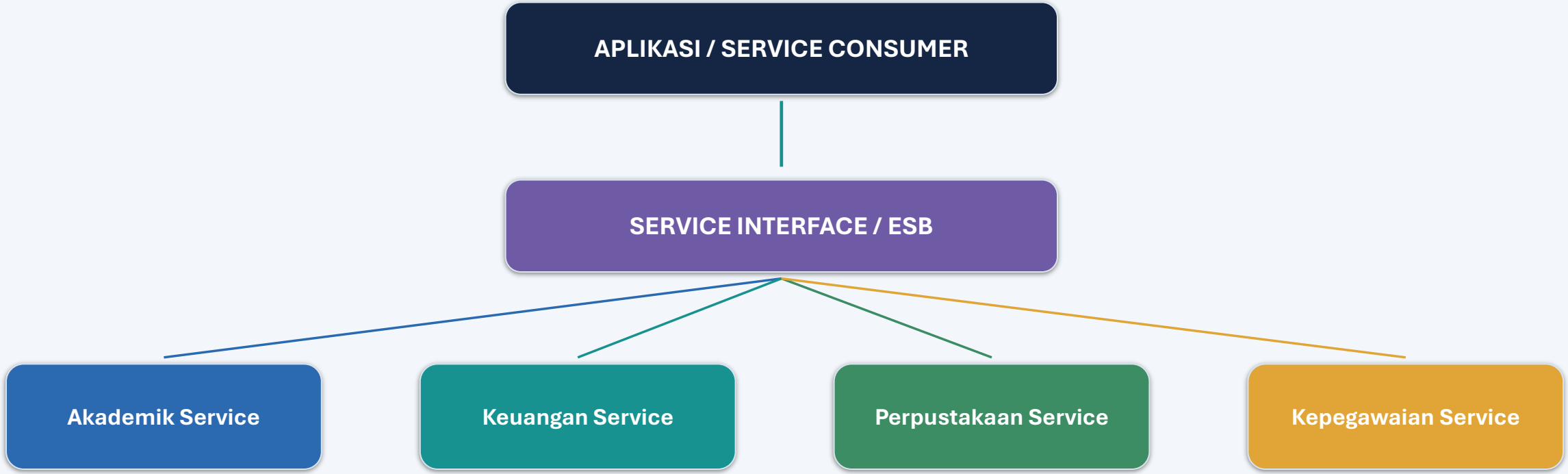
Satu event dapat ditanggapi oleh beberapa consumer



Keunggulan: komponen lebih longgar keterikatannya dan mendukung proses yang tidak harus serentak.

Service-Oriented Architecture (SOA)

Menyusun sistem berdasarkan layanan yang dapat digunakan bersama



SOA mendukung interoperabilitas, penggunaan ulang layanan, dan integrasi sistem. ESB dapat membantu mengatur komunikasi.

Contoh SOA: Integrasi Sistem Universitas

Layanan dari beberapa sistem bekerja sama dalam satu proses



Sistem akademik menggunakan layanan keuangan untuk memeriksa pembayaran. Masing-masing sistem tetap memiliki fungsi dan pengelolaan sendiri.

SOA berguna ketika organisasi perlu mengintegrasikan aplikasi yang telah ada.

SOA vs Microservices

Keduanya berorientasi layanan, tetapi fokusnya tidak sama

SOA

- Fokus integrasi layanan enterprise
- Dapat menghubungkan banyak sistem organisasi
- Dapat menggunakan ESB
- Menekankan interoperabilitas dan reuse

MICROSERVICES

- Fokus layanan kecil dan independen
- Umumnya berada dalam satu aplikasi/domain
- API dan messaging sering digunakan
- Menekankan independensi deployment dan skalabilitas

Keduanya dapat saling melengkapi dalam arsitektur perusahaan.

Desain Arsitektur Cloud-Native

Aplikasi dirancang untuk memanfaatkan kemampuan lingkungan cloud

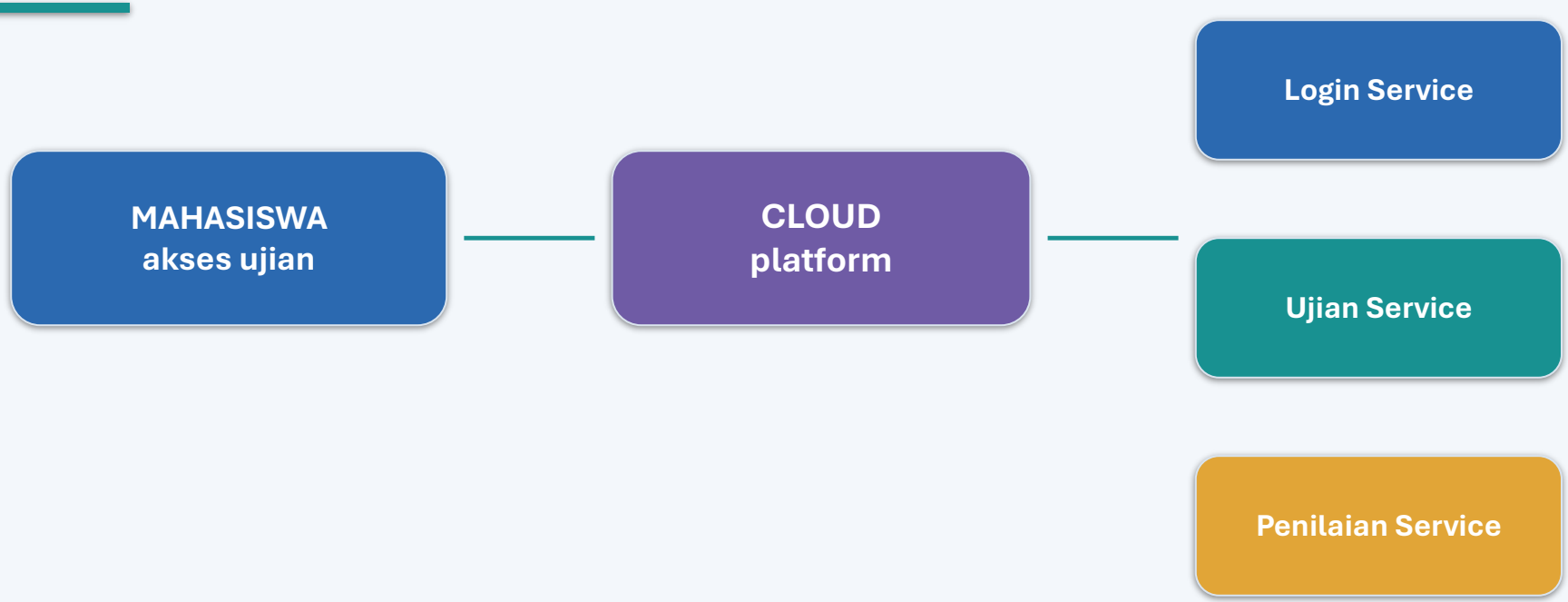
MICROSERVICES	CONTAINERS	ORCHESTRATION	CI/CD	SCALABILITY
Layanan dengan fungsi spesifik	Aplikasi dan dependensi dikemas bersama	Pengelolaan container otomatis	Integrasi, pengujian, delivery otomatis	Kapasitas menyesuaikan beban

Cloud-native menekankan otomatisasi, skalabilitas, ketahanan, dan pengelolaan aplikasi secara modern.

Teknologi contoh: Docker untuk container dan Kubernetes untuk orkestrasi container.

Contoh Cloud-Native: LMS Universitas

Kapasitas layanan dapat disesuaikan ketika beban meningkat



Beban ujian meningkat → kapasitas layanan ditambah sesuai kebutuhan.

Catatan: aplikasi yang sekadar di-hosting di cloud belum tentu cloud-native. Desain dan operasinya perlu memanfaatkan karakteristik cloud.

Cloud-Based vs Cloud-Native

Lokasi hosting tidak otomatis menentukan gaya arsitektur

CLOUD-BASED

Aplikasi dijalankan pada infrastruktur cloud.

Bisa berupa aplikasi monolith atau aplikasi dengan desain lain.

CLOUD-NATIVE

Aplikasi dirancang untuk memanfaatkan cloud.

Umumnya menekankan otomasi, skalabilitas, container, dan ketahanan.

Cloud-native adalah pendekatan desain dan operasi, bukan sekadar tempat menjalankan aplikasi.

Evaluasi Arsitektur Perangkat Lunak

Menilai kesesuaian rancangan dengan kebutuhan sistem

Performance

Kecepatan / waktu respons

Scalability

Kemampuan menangani kenaikan beban

Security

Perlindungan data dan akses

Reliability

Keandalan operasi

Maintainability

Kemudahan pemeliharaan

Availability

Ketersediaan layanan

Evaluasi menguji apakah arsitektur mendukung kebutuhan fungsional dan nonfungsional.

Metode: ATAM • SAAM • Performance Testing • Security Testing

Metode Evaluasi Arsitektur

Menggunakan pendekatan yang sesuai dengan pertanyaan evaluasi

ATAM

Architecture Tradeoff Analysis Method

Menilai karakteristik kualitas dan konsekuensi pilihan desain.

SAAM

Software Architecture Analysis Method

Menganalisis kemampuan arsitektur menghadapi perubahan dan skenario.

PENGUJIAN

Performance dan Security Testing

Mengukur kinerja serta mengidentifikasi kelemahan keamanan.

Hasil evaluasi menjadi dasar perbaikan, bukan sekadar pemberian nilai lulus atau gagal.

Contoh Hasil Evaluasi Sistem Akademik

Ilustrasi saat banyak mahasiswa mengakses KRS

Indikator	Target	Hasil	Kesimpulan
Waktu respons	≤ 3 detik	2 detik	Memenuhi
Availability	≥ 99%	98%	Perbaiki
Error transaksi	≤ 1%	0,5%	Memenuhi
Keamanan	Lolos uji	Ada celah	Perbaiki

Angka hanya ilustrasi; target ditetapkan berdasarkan kebutuhan dan kesepakatan layanan.

Dokumentasi Arsitektur

Mencatat struktur, hubungan, keputusan, dan teknologi sistem

Architecture Diagram	Gambaran komponen dan hubungan
Component Diagram	Komponen dan dependensi
Deployment Diagram	Penempatan sistem pada infrastruktur
DFD	Aliran data
Sequence Diagram	Urutan pesan dan interaksi

Dokumentasi membantu programmer, analis, penguji, administrator, dan tim pemeliharaan memahami sistem.

C4 Model: Dokumentasi Bertingkat

Memahami sistem dari konteks hingga detail kode



Sistem, pengguna, dan sistem eksternal

Aplikasi, database, layanan utama

Komponen di dalam container

Struktur kode dan detail implementasi

C4 memberi tingkat detail berbeda: pemangku kepentingan melihat konteks, tim teknis menelusuri container, komponen, hingga kode.

Studi Kasus Terintegrasi: Toko Online

Gabungkan prinsip, model arsitektur, algoritma, dan evaluasi

- Registrasi dan login
- Katalog dan pencarian produk
- Keranjang dan pemesanan
- Pembayaran dan pembaruan stok
- Pengiriman dan notifikasi
- Laporan transaksi



Komunikasi dapat memakai API; event memicu proses stok, pengiriman, dan notifikasi.

Aktivitas / Tugas Analisis Mahasiswa

Pilih satu sistem dan buat rancangan arsitektur ringkas

PILIH OBJEK

E-commerce • Sistem akademik • LMS • Perbankan • Transportasi online

Identifikasi pengguna, fungsi utama, dan kebutuhan kualitas sistem.

HASIL YANG DIKUMPULKAN

1. Model arsitektur dan alasan pemilihan
2. Diagram komponen dan aliran data
3. Algoritma proses utama
4. Tiga indikator evaluasi

Gunakan Draw.io, PlantUML, atau alat pemodelan lain untuk membuat diagram.

KESIMPULAN

Arsitektur yang baik memastikan komponen tersusun jelas, berinteraksi secara tepat, dan memenuhi kebutuhan kualitas sistem.

RANCANGKAN

IMPLEMENTASIKAN

EVALUASIKAN

Algoritma menjelaskan bagaimana proses diselesaikan; arsitektur menjelaskan bagaimana komponen disusun dan berinteraksi.

Terima kasih • Diskusi dan tanya jawab